

Lab 9 IPC - Questions & Answers

Author: Sharique 28369

Section A: Pipes and FIFO

Task A1: Bidirectional Pipe

Q1. Why does the parent close `fd[0]` and the child close `fd[1]`? A: In a unidirectional pipe, `fd[0]` is the read end and `fd[1]` is the write end. Closing the unused ends prevents resource leaks and ensures that processes don't hang waiting for data from themselves or for a write end that is never closed. For example, a `read()` only returns 0 (EOF) when all write ends of a pipe are closed.

Q2. What happens if you do not close the unused ends of the pipe? A: The program may encounter several issues:

1. **Hanging/Deadlock:** A `read()` call might wait indefinitely because the process itself still has a write end open, preventing an EOF from being signaled.
2. **Resource Exhaustion:** Each open file descriptor consumes system resources. In large loops, this can hit the process limit.
3. **Unexpected Behavior:** If a process accidentally writes to or reads from the "wrong" end, data integrity is lost.

Q3. Can `pipe()` be used between two unrelated processes? Why or why not? A: No. `pipe()` creates an anonymous (unnamed) pipe that only exists in the memory of the creating process and its children. Unrelated processes have no way of "finding" the file descriptors for an anonymous pipe. FIFOs (Named Pipes) are used for unrelated processes.

Task A2: Shell Pipeline

Q1. What is the role of `dup2(fd[1], STDOUT_FILENO)`? A: It duplicates the file descriptor `fd[1]` (the write end of the pipe) onto the standard output (`STDOUT_FILENO`). This causes any subsequent output from the process (like from `ls`) to be sent into the pipe instead of the screen.

Q2. Why must original file descriptors be closed after `dup2()`? A: After `dup2()`, the process has two file descriptors referring to the same pipe end (e.g., both `fd[1]` and `1`). Closing the original `fds` ensures that the only ways to access the pipe are via the standard streams, and it allows the pipe to be fully closed when the process finishes or redirects again, which is crucial for signaling EOF to the reader.

Q3. How would you extend this to simulate: `cat file.txt | grep "error" | wc -l`? A: You would need two pipes.

1. Create Pipe A and Pipe B.
2. Fork Child 1: Redirect `stdout` to Pipe A write-end, execute `cat`.
3. Fork Child 2: Redirect `stdin` to Pipe A read-end AND `stdout` to Pipe B write-end, execute `grep`.
4. Fork Child 3: Redirect `stdin` to Pipe B read-end, execute `wc`.
5. Parent closes all ends and waits.

Task A3: Two-Way FIFO Communication

Q1. What is the key difference between unnamed pipes and FIFOs? A: Unnamed pipes are temporary and only for related processes. FIFOs (Named Pipes) exist as files in the filesystem, have names and permissions, and can be used by any process that has access to that file path, regardless of relationship.

Q2. What permissions does 0666 grant in mkfifo()? Who can override them? A: 0666 grants read and write permissions to the Owner, Group, and Others. However, these are modified by the process's `umask` (user file-creation mode mask). The system administrator (root) can also override or change these permissions.

Q3. What happens if the reader process opens the FIFO before the writer? A: By default, `open()` for a FIFO is a blocking operation. If a process opens it for reading (`O_RDONLY`), it will block until another process opens it for writing (`O_WRONLY`), and vice versa.

Section B: Shared Memory

Q1. What is the purpose of ftok() and how does it generate a key? A: `ftok()` (File to Key) converts a pathname and a project identifier (an integer) into a `key_t` for System V IPC. It usually combines the file's inode number and the provided ID to generate a reasonably unique key that unrelated processes can agree upon to access the same IPC resource.

Q2. What happens if shmctl(shmid, IPC_RMID, NULL) is omitted and the process exits? A: System V Shared Memory segments are kernel-persistent. If not explicitly removed, the memory remains allocated in the kernel even after the processes exit. It will only be removed when the system reboots or when manually removed using the `ipcrm` command.

Q3. Why is sleep(1) in the starter code a fragile synchronisation strategy? A: `sleep()` is a "race condition" band-aid. It doesn't guarantee that the parent has actually finished writing; it just hopes it has. If the system is under heavy load, the child might wake up before the parent writes, or the parent might take longer than 1 second. Proper synchronization requires Semaphores or Mutexes.

Q4. How does shared memory compare to pipes in terms of speed and data persistence? A:

- **Speed:** Shared memory is significantly faster because it avoids copying data between user space and kernel space (unlike pipes which require `write()/read()` syscalls).
 - **Persistence:** Shared memory is kernel-persistent (stays until deleted), whereas pipe data is transient and exists only as long as there are open file descriptors.
-

Section C: Sockets

Task C1: Socket between Forked Processes

Q1. How does socketpair() differ from pipe() in terms of directionality? A: `pipe()` is unidirectional (one read end, one write end). `socketpair()` creates a pair of connected, full-duplex sockets; both ends can be used for both reading and writing simultaneously.

Q2. What socket domain and type are appropriate here? Why? A:

- **Domain:** `AF_UNIX` (or `AF_LOCAL`) because the communication is local to the same host.

- **Type:** `SOCK_STREAM` for reliable, sequenced, two-way byte streams (TCP-like behavior) or `SOCK_DGRAM` for datagrams. `SOCK_STREAM` is typical for general IPC.

Task C2: Socket between Unrelated Processes

Q1. Why must the server call `bind()`, `listen()`, and `accept()` before communicating? A:

- `bind()`: Assigns a name (path) to the socket so clients can find it.
- `listen()`: Tells the kernel the socket is ready to receive incoming connection requests.
- `accept()`: Extracts the first connection request from the queue and creates a new socket specific to that client for communication.

Q2. What must be done with the socket file after the server exits? A: The socket file remains in the filesystem. It should be removed (using `unlink()`) before starting the server again, or the next `bind()` call will fail with "Address already in use".

Task C3: Datagram Sockets with Multiple Clients

Q1. What is lost when switching from `SOCK_STREAM` to `SOCK_DGRAM`? A: You lose connection-oriented features: reliability (messages can be lost), ordering (messages can arrive out of order), and boundary control (stream vs. discrete packets). You also lose the byte-stream abstraction.

Q2. How does the server identify which client to reply to when using `recvfrom()`? A: `recvfrom()` takes a `sockaddr` structure as an argument. The kernel populates this structure with the address (path in `AF_UNIX` or IP/Port in `AF_INET`) of the source client, which the server can then use in `sendto()`.

Section D: Message Queues

Q1. Why must `mtype` be a positive long in System V message queues? A: The `mtype` field is used by the kernel to categorize messages. A positive `long` allows the receiver to select specific types of messages (using the `msgtyp` argument in `msgrcv()`). Values like 0 or negative integers have special meanings in the `msgrcv()` API (receive any message or priority-based receiving).

Q2. What happens to unreceived messages if the queue is deleted mid-run? A: If `msgctl(IPC_RMID)` is called, the queue and all its contained messages are immediately destroyed. Any process blocked on `msgrcv()` or `msgsnd()` will return an error (`EIDRM`).

Q3. (Task D2) What `errno` value is set when `IPC_NOWAIT` finds no message? How should it be handled? A: `ENOMSG`. It should be handled by checking `if (errno == ENOMSG)` and potentially retrying later, polling, or switching to a blocking wait if the message is critical.